

Vendor and Purchase Order Manager - Complete Project Guide

This guide explains the project in detail for learning, demo preparation, and interview or viva discussion. It covers the application architecture, backend, frontend, database, authentication, deployment, CI/CD, Docker, and Jenkins.

1. Project Overview

Vendor and Purchase Order Manager is a full-stack MERN application for procurement and vendor operations. It helps an organization manage vendors, purchase orders, invoices, payments, budgets, contracts, inventory, audit logs, notifications, and analytics.

The project is built like a real business application. It has secure login, role-based access, dashboards, reports, background jobs, file import/export support, live notifications, API documentation, and deployment automation.

Main users:

User Type	Main Responsibility
Admin	Manages the full system, users, audit logs, and configuration
Manager	Manages vendors, purchase orders, contracts, inventory, and operational workflows
Accountant	Manages invoices, payments, budgets, accounting, and finance-related records
Viewer	Reads allowed information without changing important data

Procurement lifecycle:

Vendor
Purchase Order
Approval
Delivery
Invoice
Payment
Reporting
Audit Log

2. Technology Stack

Backend

Technology	Purpose
Node.js	JavaScript runtime for the backend
Express.js	HTTP server and routing framework
Mongoose	MongoDB object modeling and schema validation
JSON Web Token	Stateless authentication after login
bcryptjs	Password hashing and password comparison
cors	Allows the frontend domain to call backend APIs
dotenv	Loads environment variables
express-rate-limit	Limits repeated API calls
multer	Handles file uploads
exceljs	Generates Excel exports
csv-parser	Reads CSV imports
nodemailer	Sends emails
socket.io	Supports real-time notifications
node-cron	Runs scheduled backend jobs
swagger-jsdoc and swagger-ui-express	Generates API documentation

Frontend

Technology	Purpose
React	Builds the user interface
Vite	Development server and production build tool

Tailwind CSS	Utility-based styling
React Router	Client-side page routing
Axios	HTTP client for API requests
Recharts	Charts and dashboard visualizations
Socket.io Client	Receives real-time events from backend
React Hot Toast	Displays success and error messages
Lucide React	Icon library used inside the UI

DevOps And Deployment

Tool	Purpose
GitHub	Source code hosting
GitHub Actions	Primary CI/CD automation
Jenkins	Optional CI/CD pipeline for enterprise-style workflow
Docker	Container packaging and local service setup
Docker Compose	Runs frontend, backend, and MongoDB together locally
Render	Hosts the backend service
Vercel	Hosts the frontend application
MongoDB Atlas	Hosts the production database

3. Folder Structure

```
vendor-system/  
  client/  
    src/  
      components/  
      context/  
      lib/  
      pages/  
      App.jsx  
      main.jsx  
      index.css  
    Dockerfile  
    nginx.conf  
    package.json  
    vite.config.js  
    vercel.json  
  server/  
    config/  
      db.js  
      swagger.js  
    controllers/  
    middleware/  
      auth.js  
      rbac.js  
      errorHandler.js  
      rateLimiter.js  
    models/  
    routes/  
    utils/  
    Dockerfile  
    package.json  
    seed.js  
    server.js  
  .github/  
    workflows/  
      ci.yml  
      cd.yml
```

DEPLOYMENT_SECRETS.md

Jenkinsfile

docker-compose.yml

render.yaml

README.md

SYSTEM_DESIGN.md

PROJECT_GUIDE.md

4. How To Run Locally

Backend Setup

cd server

npm install

Create `server/.env`:

MONGODB_URI=mongodb://localhost:27017/vendor_po_manager

JWT_SECRET=your-local-secret

PORT=5000

NODE_ENV=development

FRONTEND_URL=http://localhost:5173

ALLOW_PUBLIC_REGISTRATION=true

Seed demo data:

node seed.js

Start backend:

node server.js

Backend runs on:

http://localhost:5000

Frontend Setup

cd client

npm install

Create `client/.env`:

VITE_API_URL=http://localhost:5000/api

Start frontend:

npm run dev

Frontend runs on:

http://localhost:5173

Demo Login

| Role | Email | Password |

| --- | --- | --- |

| Admin | admin@vendor.com | admin123 |

5. Backend Architecture

The backend starts from `server/server.js`.

Startup flow:

Load environment variables

Connect to MongoDB

Create Express app

Create HTTP server
Attach Socket.io
Configure CORS
Configure JSON body parsing
Configure rate limiting
Register static uploads
Register Swagger documentation
Register API routes
Register error handlers
Start cron jobs
Start listening on PORT

Database Connection

The database connection is handled in:

server/config/db.js

The backend reads:

MONGODB_URI

or the older supported name:

MONGO_URI

If no database connection string exists, the backend stops because the application cannot safely run without a database.

Route Structure

Routes are stored in `server/routes`. Controllers are stored in `server/controllers`.

Route files define API paths. Controller files contain business logic.

Example flow:

/api/vendors request
vendorRoutes.js matches route
auth middleware verifies token
rbac middleware checks role
vendorController.js runs logic
Vendor.js model queries MongoDB
JSON response returns to frontend

6. Frontend Architecture

The frontend is a React application built with Vite.

Important files:

File	Purpose
client/src/main.jsx	React entry point
client/src/App.jsx	Route definitions
client/src/context/AuthContext.jsx	Stores login state and user permissions
client/src/context/ThemeContext.jsx	Stores theme state
client/src/lib/api.js	Axios API client
client/src/components/Layout.jsx	Main protected application layout
client/src/components/ProtectedRoute.jsx	Prevents unauthenticated access

Frontend request flow:

Page component needs data
Component calls API helper
Axios sends request to VITE_API_URL
Token is sent in Authorization header
Backend returns JSON

Frontend updates state
UI re-renders

7. Authentication

Authentication is based on JWT.

Login flow:

User enters email and password
Frontend sends POST /api/auth/login
Backend finds user by email
Backend compares entered password with hashed password
Backend creates JWT token
Frontend stores token and user details
Frontend redirects user to dashboard

Protected request flow:

Frontend sends Authorization: Bearer token
Backend verifies token using JWT_SECRET
Backend finds user from token id
Backend attaches user to request
Controller handles the request

Passwords are not stored as plain text. The `User` model hashes passwords using bcrypt before saving.

8. Authorization And Roles

The project uses Role-Based Access Control.

Role checking happens in backend middleware. The frontend may hide pages, but backend authorization is the real protection.

Role	Typical Access
Admin	Full system access
Manager	Vendor, purchase order, contract, inventory, budget, and analytics workflows
Accountant	Invoice, payment, budget, accounting, and finance workflows
Viewer	Read-only access where allowed

If a user does not have permission, the backend returns an authorization error.

9. Main Features

Vendor Management

Vendors store contact details, GST or tax information, item prices, rating, performance score, and risk information. Vendor data is used by purchase orders, contracts, invoices, inventory, and analytics.

Purchase Orders

Purchase orders store the vendor, line items, quantity, price, total amount, department, status, approval status, and recurring information. The workflow supports draft, submitted, approved, rejected, delivered, and cancelled states depending on the controller logic.

Invoices And Payments

Invoices track amount, paid amount, outstanding amount, due date, and payment status. Payments update invoice status and help finance reports show outstanding or overdue amounts.

Dashboard

The dashboard summarizes key business information such as total vendors, purchase orders, pending payments, overdue payments, purchase trends, and vendor performance.

Analytics

Analytics includes payment aging, vendor spend, monthly growth, vendor reliability, forecasting, anomaly detection, compliance reports, and vendor comparison.

Inventory

Inventory tracks stock items, current stock, reorder point, reorder quantity, and preferred vendor. Scheduled jobs can check inventory levels and help create reorder workflows.

Contracts

Contracts track vendor agreements, start date, end date, value, status, and renewal reminders.

Audit Logs

Audit logs record important user actions. This is useful for compliance, debugging, and accountability.

Notifications

Notifications inform users about important events. Socket.io allows the frontend to receive updates without refreshing.

10. API Reference Summary

Method	Type	Route	Area	Purpose
POST		/api/auth/login`	Login and receive JWT token	
GET/POST/PUT/DELETE		/api/vendors`	Vendor CRUD	
GET/POST/PUT/DELETE		/api/purchase-orders`	Purchase order CRUD and workflow	
GET/POST/PUT/DELETE		/api/invoices`	Invoice management	
GET/POST		/api/payments`	Payment records	
GET		/api/dashboard`	Dashboard stats	
GET		/api/analytics`	Reports and charts	
GET/POST/PUT/DELETE		/api/budgets`	Budget management	
GET		/api/audit-logs`	Audit log viewing	
GET/PUT		/api/notifications`	Notification viewing and read status	
POST		/api/import`	CSV import	
GET		/api/export`	Excel export	
GET/POST/PUT/DELETE		/api/contracts`	Contract management	
GET/POST/PUT/DELETE		/api/inventory`	Inventory management	
GET/POST/PUT/DELETE		/api/webhooks`	Webhook configuration	

Swagger documentation is available locally at:

<http://localhost:5000/api/docs>

11. Environment Variables

Backend Variables

Variable	Purpose
MONGODB_URI`	MongoDB connection string
JWT_SECRET`	Secret used to sign and verify JWT tokens
PORT`	Backend server port
NODE_ENV`	Environment mode
FRONTEND_URL`	Allowed frontend URL for CORS and links
ALLOW_PUBLIC_REGISTRATION`	Enables or disables public signup
SMTP_HOST`	Email server host
SMTP_PORT`	Email server port
SMTP_USER`	Email username
SMTP_PASS`	Email password
GOOGLE_CLIENT_ID`	Google OAuth client id
GOOGLE_CLIENT_SECRET`	Google OAuth secret

Frontend Variables

Variable	Purpose
----------	---------

```
| --- | --- |  
| `VITE_API_URL` | Backend API base URL |  
| `VITE_GOOGLE_CLIENT_ID` | Optional Google login client id |
```

12. Production Deployment

Production deployment uses:

Vercel for frontend
Render for backend
MongoDB Atlas for database

Render Backend

Render uses `render.yaml`.

Build command:

```
cd server && npm ci
```

Start command:

```
cd server && node server.js
```

Important Render variables:

```
MONGODB_URI  
JWT_SECRET  
NODE_ENV=production  
FRONTEND_URL=https://your-vercel-app.vercel.app  
ALLOW_PUBLIC_REGISTRATION=false
```

Vercel Frontend

Vercel builds the frontend from the `client` directory.

Important Vercel variable:

```
VITE_API_URL=https://your-render-service.onrender.com/api
```

13. Docker

Docker Compose starts the full stack locally:

```
docker compose up --build
```

Services:

```
| Service | Port | Purpose |  
| --- | --- | --- |  
| mongodb | 27017 | Local database |  
| server | 5000 | Backend API |  
| client | 3000 | Frontend served by Nginx |
```

The backend Dockerfile creates a Node.js image. The frontend Dockerfile builds the Vite app and serves the static output with Nginx.

Docker is also checked in CI to confirm the project can be containerized.

14. GitHub Actions CI

CI means Continuous Integration. It checks whether new code is healthy before deployment.

Workflow file:

```
.github/workflows/ci.yml
```

Trigger:

Push to main
Push to develop

Pull request to main

CI jobs:

Server Checks

The server job:

1. Starts MongoDB in GitHub Actions.
2. Installs backend dependencies with ``npm ci``.
3. Checks backend syntax with ``node --check server.js``.
4. Seeds the CI database with ``node seed.js``.
5. Starts the backend on port ``5050``.
6. Tests ``/api/health``.
7. Logs in with seeded admin credentials.
8. Extracts the JWT token.
9. Calls the authenticated dashboard API.

This confirms the backend can install, connect to MongoDB, start, authenticate, and serve protected data.

Client Build

The client job:

1. Installs frontend dependencies with ``npm ci``.
2. Builds the frontend with ``npm run build``.

This confirms the frontend can build for production.

Docker Build Test

The Docker job:

1. Validates ``docker-compose.yml``.
2. Builds the backend Docker image.
3. Builds the frontend Docker image.

This confirms the Docker setup is valid.

15. GitHub Actions CD

CD means Continuous Deployment. It deploys code after CI passes.

Workflow file:

`.github/workflows/cd.yml`

CD starts after the CI workflow completes. It deploys only if:

CI result is success

branch is main

Required GitHub repository secrets:

`RENDER_DEPLOY_HOOK_URL`

`VERCEL_DEPLOY_HOOK_URL`

CD calls these deploy hooks with ``curl``. Render and Vercel then pull the latest code and redeploy.

Secrets must be stored in GitHub settings, not in code:

Settings

Secrets and variables

Actions

New repository secret

16. Jenkins

Jenkins is an optional alternative CI/CD system for this project.

Pipeline file:

Jenkinsfile

Jenkins pipeline stages:

1. Install backend dependencies.
2. Check backend syntax.
3. Install frontend dependencies.
4. Build frontend.
5. Trigger Render and Vercel deploy hooks on the `main` branch.
6. Archive frontend build files.

Jenkins credentials needed:

render-deploy-hook-url

vercel-deploy-hook-url

Jenkins workflow:

Developer pushes code to GitHub

GitHub webhook calls Jenkins

Jenkins pulls latest code

Jenkins reads Jenkinsfile

Jenkins runs pipeline stages

Jenkins triggers Render and Vercel deploy hooks

Live app updates

If Jenkins is used as the main deployment system, GitHub Actions deployment should be disabled to avoid duplicate deployments.

17. GitHub Actions Compared With Jenkins

Topic	GitHub Actions	Jenkins
---	---	---
Hosting	Built into GitHub	Requires Jenkins server
Setup	Easier for GitHub repositories	More setup and maintenance
Secrets	GitHub repository secrets	Jenkins credentials
Pipeline file	YAML workflow files	Jenkinsfile
Best for this project	Simple primary CI/CD	Optional enterprise-style learning
Deployment risk if both are active	May duplicate deployment	May duplicate deployment

Recommended explanation:

GitHub Actions is the primary CI/CD pipeline because it is integrated with GitHub and simple to operate.

Jenkinsfile is included to demonstrate how the same project can be moved to an enterprise Jenkins pipeline. In production we would choose one deployment orchestrator to avoid duplicate deployments.

18. Full Workflow When Code Is Updated

GitHub Actions workflow:

Developer updates code

Developer commits changes

Developer pushes to main

GitHub starts CI

Backend dependencies install

MongoDB test service starts

Backend syntax check runs

Seed script runs

Health API is tested

Login API is tested

Dashboard API is tested with JWT

Frontend dependencies install

Frontend production build runs

Docker config and images are checked

CI passes

GitHub starts CD
CD reads GitHub secrets
CD triggers Render backend hook
CD triggers Vercel frontend hook
Render redeploys backend
Vercel redeploys frontend
Live project updates

Jenkins workflow:

Developer updates code
Developer pushes to GitHub
GitHub webhook triggers Jenkins
Jenkins pulls latest code
Jenkins installs backend dependencies
Jenkins checks backend syntax
Jenkins installs frontend dependencies
Jenkins builds frontend
Jenkins reads deploy hook credentials
Jenkins triggers Render and Vercel
Live project updates

19. Common Issues

Frontend Cannot Reach Backend

Check Vercel environment variable:

VITE_API_URL=https://your-render-service.onrender.com/api

Check Render environment variable:

FRONTEND_URL=https://your-vercel-app.vercel.app

CORS Error

The backend is rejecting the frontend origin. Set `FRONTEND\_URL` in Render to the exact Vercel frontend URL.

Backend Cannot Connect To MongoDB

Check `MONGODB\_URI`, MongoDB Atlas username/password, and Atlas network access.

CI Fails At Seed Script

Check that the GitHub Actions MongoDB service started correctly and that `MONGODB\_URI` points to the CI MongoDB instance.

Frontend Build Fails

Check imports, missing files, package versions, and Vite environment variables.

CD Does Not Deploy

Check that GitHub repository secrets exist:

RENDER_DEPLOY_HOOK_URL

VERCEL_DEPLOY_HOOK_URL

Open GitHub Actions logs and look for:

Render deploy hook triggered.

Vercel deploy hook triggered.

Docker Build Fails Locally

Make sure Docker Desktop is running. Local Docker errors can happen even when GitHub Actions Docker checks work.

20. Demo Explanation

Short version:

This is a full-stack MERN procurement system. The React frontend runs on Vercel, the Express backend runs on Render, and MongoDB Atlas stores data. The backend uses JWT authentication and role-based access control. GitHub Actions validates the backend, frontend, and Docker setup, then triggers Render and Vercel deployment hooks after CI passes. A Jenkinsfile is also included to show how the same workflow can be implemented in Jenkins.

Detailed version:

When code is pushed to main, GitHub Actions starts the CI pipeline. The server job installs backend dependencies, starts MongoDB, checks syntax, seeds data, starts the API, tests the health endpoint, logs in, extracts a JWT token, and calls an authenticated dashboard route. The client job installs frontend dependencies and builds the Vite app. The Docker job validates Docker Compose and builds both images. If all checks pass, the CD pipeline starts and calls deploy hooks stored in GitHub Secrets. Render redeploys the backend and Vercel redeploys the frontend. Jenkins is available as an optional alternative CI/CD path through the Jenkinsfile.

21. Glossary

Term	Meaning
---	---
MERN	MongoDB, Express, React, Node.js
API	Interface used by frontend to communicate with backend
REST	HTTP-based API design using GET, POST, PUT, DELETE
JWT	Token used to prove a user is logged in
RBAC	Role-Based Access Control
CI	Continuous Integration, used to test code automatically
CD	Continuous Deployment, used to deploy code automatically
Deploy Hook	Secret URL that starts deployment on a hosting platform
CORS	Browser security rule controlling cross-origin requests
Docker	Tool for packaging applications into containers
Jenkins	CI/CD automation server
GitHub Actions	CI/CD automation built into GitHub